

# SpectralGraph

A NoSQL-Backed Red Team Asset & Vulnerability Tracker

**Zuni**

BSc Computer Science (4th Semester)  
Abbottabad University of Science & Technology

Project Defense **2026**

# 1. The Problem: Manual Tracking Infrastructures

The exponential growth of unstructured network data renders manual tracking via static spreadsheets obsolete.

## Critical Structural Failures

- **Loss of Historical Context:** Overwriting previous scan data destroys the audit trail of the network's temporal state.
- **Inefficient Normalization:** Flat files cannot efficiently map complex, many-to-many relationships between targets and CVEs.
- **Inadequate Access Controls:** Spreadsheets lack application-layer security, compromising data integrity.

## 2. The Solution: SpectralGraph

An automated, NoSQL-backed database pipeline designed to bridge offensive cybersecurity operations and modern data architecture.

### Automated Ingestion

Ingests raw telemetry (Nmap) directly into MongoDB, eliminating manual data entry errors.

### Relational Mapping

Securely and dynamically links specific CVEs to target infrastructure using NoSQL referencing.

### Temporal Logging

Maintains an immutable time-series audit log of infrastructure state changes.

### 3. Core Technology Stack

#### Backend Infrastructure

- **Database:** MongoDB 7.0  
(*NoSQL Document Store*)
- **Logic Tier:** Python 3.11+
- **Drivers:** PyMongo API

#### Security & Environment

- **Telemetry:** python-nmap
- **Encryption:** hashlib  
(*SHA-256 for credentials*)
- **OS:** Kali Linux / Windows

## 4. Database Architecture: Hybrid Schema

SpectralGraph utilizes two distinct NoSQL structural paradigms to optimize read throughput and data normalization:

**targets** (*Embedding*) Stores the "Current State". Open ports and services are embedded as arrays for rapid, single-query reads.

**vulnerabilities** (*Referencing*) Normalizes data by linking specific CVEs back to the host via a `target_id`.

**scan\_history** (*Referencing*) Generates an immutable, time-stamped audit trail of network discovery.

**users** (*Standalone*) Manages operator credentials and RBAC rules.

## 5. Core Feature: Idempotency & Upserts

**The Challenge:** Executing 50 scans on a single IP address must not result in 50 duplicate database records.

### PyMongo State Management

The ingestion engine utilizes MongoDB's `upsert=True` logic combined with unique field indexing:

- **Condition A (Host Exists):** The database dynamically updates the embedded port arrays to reflect the newest state.
- **Condition B (Host Missing):** The database initializes a completely new target document.

## 6. Application-Layer Security (RBAC)

Data integrity requires strict application-layer access controls. SpectralGraph enforces Role-Based Access Control (RBAC):

- **Cryptographic Storage:** Operator passwords are never stored in plaintext (SHA-256 Hashing).
- **Lead\_Operator Role:** Granted absolute write authorization to inject or modify the vulnerabilities collection.
- **Analyst Role:** Programmatically restricted to read-only operations for CVE data.

## 7. Advanced Analytical Pipelines

To generate executive summaries, the system bypasses standard read queries and executes a 4-stage MongoDB Aggregation Pipeline:

- ❶ **\$lookup**: Joins *targets* with *vulnerabilities* (NoSQL equivalent of a LEFT JOIN).
- ❷ **\$unwind**: Deconstructs vulnerability arrays to normalize documents for statistical counting.
- ❸ **\$group**: Dynamically aggregates CVEs by Operating System and categorizes by severity index.
- ❹ **\$sort**: Orders the computational output for the terminal dashboard.



**SpectralGraph successfully bridges offensive operations and modern data architecture.**

## **Project Milestones Achieved:**

- Implementation of Idempotent Data Ingestion
- Preservation of Historical Network States (Time-Series)
- Enforcement of Application Security (RBAC)
- Execution of Multi-Stage NoSQL Analytics

**Ready for Live Demonstration & Defense**